O programa
O programa lê um arquivo texto com pontos (cada ponto tem um nome/id e m coordenadas), agrupa esses pontos em k grupos usando o algoritmo de Agrupamento de Espaçamento Máximo, e escreve o resultado em outro arquivo texto. Os grupos são formados de modo que pontos próximos (distância euclidiana pequena) fiquem juntos e pontos distantes fiquem em grupos diferentes.

Entrada (arquivo texto) → vetor de Pontos → Grafo completo com distâncias → Ordena arestas por peso → Kruskal com Union-Find → Componentes conexas (k grupos) → Ordena saída → Arquivo texto.

Regras importantes

- Nenhum printf() ativo no código final — o programa não pode imprimir nada na tela (saída padrão)
- Sem variáveis globais — toda variável deve estar dentro de uma função
- Sem goto — controle de fluxo só com if / for / while
- Todo malloc() deve ter um free() correspondente — testar com valgrind
- O Makefile deve gerar o executável com o nome exato: trab1

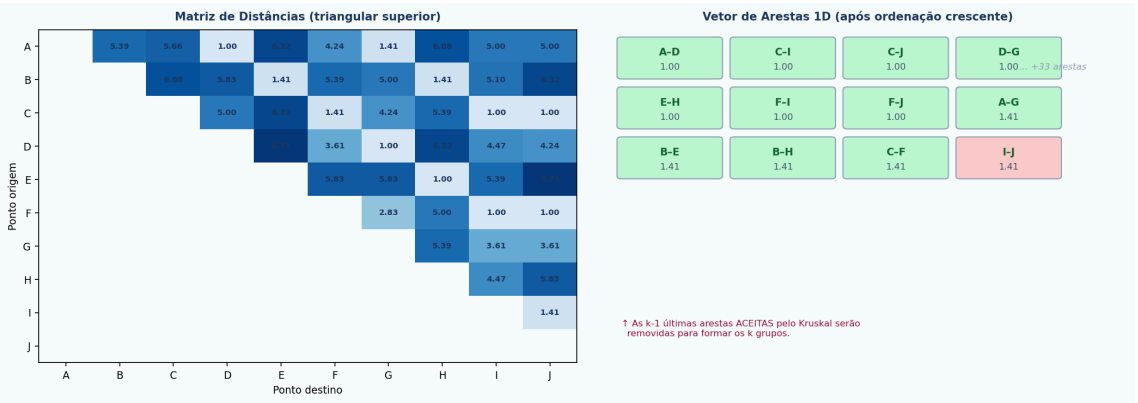
Estruturas de Dados

Point
O ponto tem nome (identificador de comprimento) e um vetor de coordenadas de tamanho m.

- point_construct() → cria um ponto alocando memória para id e coordinates
- points_distance(p1, p2) → retorna $\sqrt{\Sigma(a_i - b_i)^2}$
- sort_points(array, n) → ordena o array de pontos por nome (strcmp)
- point_destroy(p) → libera coordinate[], id e o próprio Point

Edge e Graph
A Edge guarda o peso (distância) e os índices dos dois pontos que ela conecta. O Graph guarda todos os pontos e todas as arestas possíveis. Para n pontos, há exatamente n×(n-1)/2 arestas (grafo completo, sem repetir).

Como as arestas são armazenadas em vetor de mapeamento triangular, exemplo q achei na internet:



Esquerda: matriz triangular de distâncias. Direita: vetor 1D ordenado por peso

A fórmula de mapeamento converte os índices (i, j) em uma posição do vetor:

$$\text{índice} = i \times n - (i \times (i+1)) / 2 + (j - i - 1) \quad \text{para } i < j$$

Uma matriz n×n de distâncias tem redundância: a diagonal é zero e a parte inferior é espelho da superior. Armazenamos apenas a parte superior em um vetor 1D, o que economiza quase metade da memória.

Union-Find para detectar ciclos
O Union-Find mantém dois arrays: parent[] e rank[]. Inicialmente cada ponto é seu próprio grupo (parent[i] = i). A operação find(i) retorna o representante (raiz) do grupo de i. A operação union(i, j) une os grupos de i e j.

- Compressão de caminho/Path compression : find() faz todos os nós do caminho apontarem direto para a raiz
- Union por rank: o grupo de menor "altura" é anexado sob o maior, evitando árvores compridas

Kruskal malandro, pode? Não sei
O Kruskal clássico aceita n-1 arestas para conectar todos os n pontos. Nessa forma aceitamos apenas n-k arestas, deixando k grupos desconectados. Isso evita o passo de "remover as k-1 maiores" — já paramos antes.

Kruskal processa arestas em ordem crescente. As n-k primeiras arestas aceitas são exatamente as que restariam após remover as k-1 mais pesadas da MST completa. Então parar cedo economiza trabalho sem mudar o resultado.

Para cada aresta (em ordem crescente):

se find(u) ≠ find(v) → union(u, v)

e count++

se count == n - k → PARAR

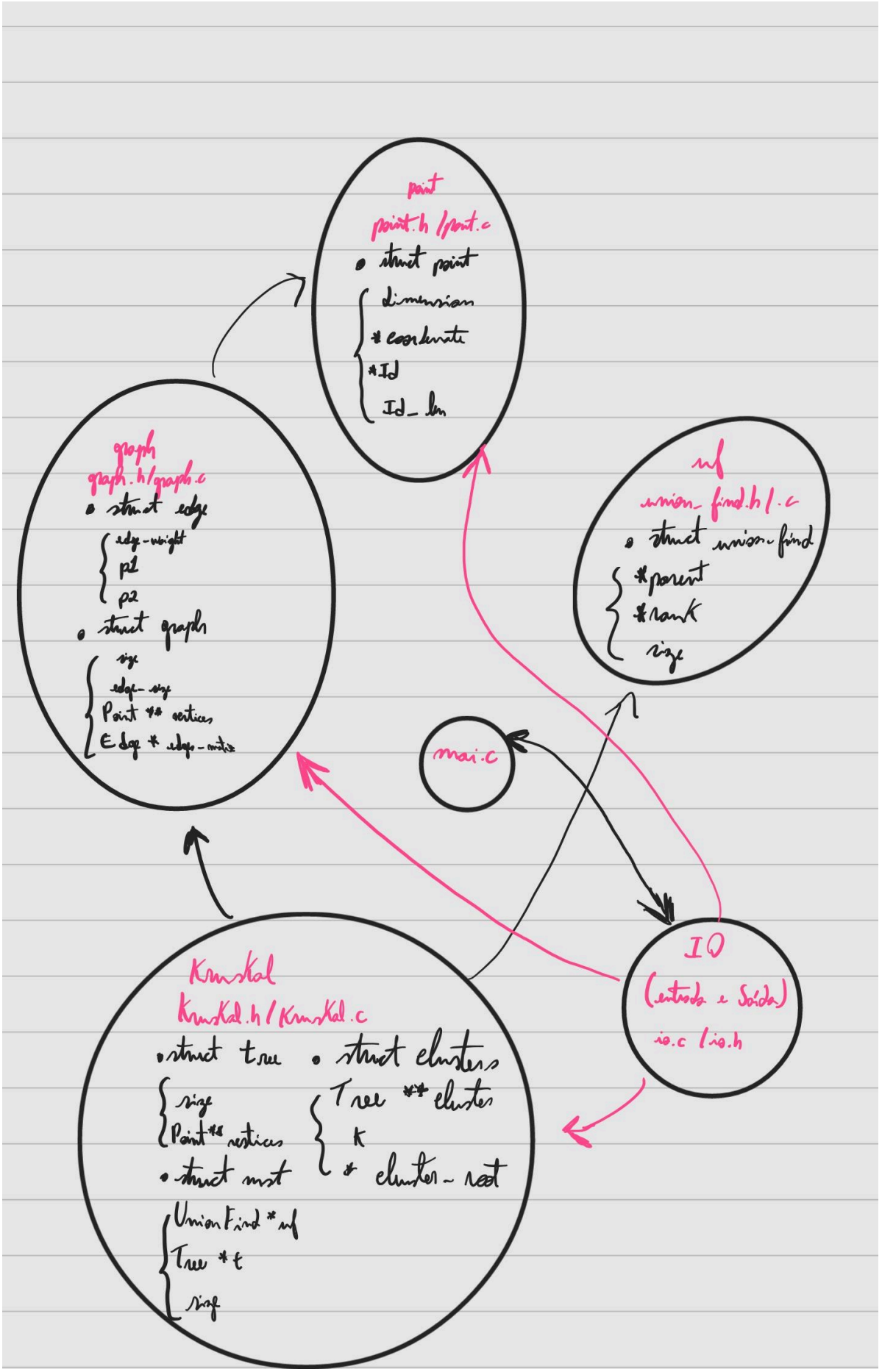
Depois do Kruskal, dois pontos estão no mesmo grupo se e somente se uf_find retorna a mesma raiz. Tipo:

- Cria um vetor cluster_of[n] e um mapa root → cluster_id.
- Para cada i em 0..n-1: root = uf_find(i). Se root é novo no mapa, atribui o próximo cluster_id. Guarda cluster_of[i] = cluster_id.
- Cria k vetores (um por cluster) e distribui os pontos.

Resumo

- **point:** abstração do ponto m-dimensional. Contém o id (string) e coordenadas. Calcula distância euclidiana e compara pontos por id.
- **graph:** armazena o vetor de pontos e o vetor de arestas do grafo completo. Calcula todas as distâncias e ordena as arestas por peso.
- **union_find:** estrutura de conjuntos disjuntos para detectar ciclos em O(α(n)) amortizado. Usa path compression e union by rank.
- **kruskal:** executa o algoritmo de Kruskal e extrai os k grupos a partir do estado final do UF.

- **io:** lê o arquivo de entrada (com getline/strtok, descobrindo m dinamicamente) e escreve o arquivo de saída no formato correto.
- **main:** valida argumentos, orquestra o fluxo completo e libera toda a memória no final.


point
point.h / point.c
• struct point
dimension
* coordinate
*Id
Id_lm
graph
graph.h/graph.c
• struct edge
edge_weight
p1
p2
• struct graph
size
edge_size
Point ** vertices
Edge * edge_matrix
uf
union_find.h / .c
• struct union_find
*parent
*rank
size
main.c
IO
(entrada e Saída)
io.c / io.h
Kruskal
Kruskal.h / Kruskal.c
• struct tree
size
Point** vertices
• struct mst
UnionFind *uf
Tree *t
size
• struct clusters
Tree ** clusters
k
* cluster_rest